

UNICESUMAR MARINGÁ

Engenharia de Software — 2º Semestre | Turma ESOFT2S-N-C

ATIVIDADE DE ESTUDO PRÁTICA — AEP

PSE EM AÇÃO — SISTEMA DE PLANEJAMENTO E
ACOMPANHAMENTO DE AÇÕES DE SAÚDE NA ESCOLA

1ª Entrega — Documento Escrito

Integrantes do grupo

Lucas Machado Feltrim Seco — RA: 26012183-2

Victor Henrique da Silva Stander — RA: 26006004-2

Fábio Henrique Soares Ferreira — RA: 26006522-2

SUMÁRIO

1 CONTEXTUALIZAÇÃO E PROBLEMA

2 OBJETIVOS

3 JUSTIFICATIVA

4 ENGENHARIA DE SOFTWARE

4.1 Usuários e delimitação do escopo

4.2 Requisitos funcionais e não funcionais

4.3 Riscos

4.4 Processo de desenvolvimento e sprints

5 ALGORITMOS E CÓDIGO

5.1 Estrutura de dados

5.2 Fluxograma geral do sistema

5.3 Fluxograma do cadastro de ação

5.4 Pseudocódigo e algoritmos em linguagem C

6 CONSIDERAÇÕES PARCIAIS

REFERÊNCIAS

1 CONTEXTUALIZAÇÃO E PROBLEMA

A escola não é apenas o lugar onde se ensinam conteúdos curriculares: é também espaço de convivência e de construção de hábitos que acompanham o estudante por toda a vida. Reconhecendo esse potencial, o Governo Federal instituiu o Programa Saúde na Escola (PSE) pelo Decreto nº 6.286, de 5 de dezembro de 2007, com a finalidade de contribuir para a formação integral dos estudantes da rede pública de educação básica por meio de ações de prevenção, promoção e atenção à saúde (BRASIL, 2007).

O programa é intersetorial: escolas, equipes pedagógicas, profissionais da Atenção Primária à Saúde e gestores municipais dividem responsabilidades no planejamento, na execução e no acompanhamento das atividades. Segundo o Ministério da Saúde, "a articulação entre Escola e Atenção Primária à Saúde é a base do Programa Saúde na Escola" (BRASIL, 2025a). Suas temáticas são amplas — alimentação saudável, atividade física, saúde bucal, vacinação, cultura de paz, saúde mental, entre outras — e cada município pactua o conjunto de ações que fará em seu território (BRASIL, 2025b).

Duas dessas temáticas foram escolhidas por este grupo como recorte da demonstração: alimentação saudável e atividade física. A Pesquisa Nacional de Saúde do Escolar mostrou que 97,3% dos estudantes de 13 a 17 anos consumiram ao menos um alimento ultraprocessado no dia anterior à pesquisa e que 61,8% foram classificados como insuficientemente ativos (IBGE, 2021). Números assim indicam que ações isoladas pouco resolvem: é preciso planejar, repetir e avaliar o alcance das atividades ao longo do tempo.

É exatamente aí que a literatura aponta a maior fragilidade do programa. Sousa, Esperidião e Medina (2017) verificaram que, embora a aproximação entre Saúde e Educação tenha fortalecido o PSE, persiste a participação desequilibrada entre os setores e a concentração de ações pontuais, frequentemente reduzidas a palestras. O material oficial do programa reforça a importância do acompanhamento ao determinar que as equipes registrem periodicamente as ações em execução e que o Grupo de Trabalho Intersetorial Municipal "deverá monitorar esse preenchimento" (BRASIL, 2011, p. 30). Na prática, porém, esse controle costuma ficar em cadernos e planilhas espalhadas entre profissionais diferentes.

A tecnologia pode organizar esse fluxo, desde que dentro de um escopo compatível com a necessidade real dos usuários. Uma aplicação simples de terminal já permite cadastrar ações, consultar o que está programado, acompanhar o andamento e gerar indicadores de participação. Ela não substitui profissionais de saúde, não emite diagnósticos e não guarda dados clínicos individuais — limite que é ético e jurídico, alinhado ao princípio da necessidade da Lei Geral de Proteção de Dados Pessoais (BRASIL, 2018). Neste trabalho serão usados apenas dados fictícios e coletivos sobre as ações.

Equipes intersetoriais que acompanham várias escolas convivem com registros dispersos e sem consolidação automática. Com isso, o planejamento perde rastreabilidade: não se sabe quantas ações foram programadas, quantas se concretizaram, quantas foram canceladas nem qual a diferença entre o público previsto e o efetivamente atendido. Daí a questão que orienta este trabalho:

Como uma aplicação desenvolvida em linguagem C pode auxiliar uma equipe escolar e de saúde a planejar, registrar e acompanhar ações coletivas do Programa Saúde na Escola, apresentando informações claras e preservando a privacidade dos estudantes?

2 OBJETIVOS

Objetivo geral

Desenvolver uma aplicação em linguagem C, executável em terminal, que apoie o planejamento, o registro e o acompanhamento de ações coletivas do Programa Saúde na Escola, sem armazenar qualquer dado pessoal ou clínico de estudantes.

Objetivos específicos

- Especificar os requisitos funcionais e não funcionais do sistema, delimitando escopo, usuários e limites éticos.
- Modelar a lógica da solução por meio de fluxogramas e de algoritmos em linguagem C que representem o menu e as seis funcionalidades mínimas.
- Planejar o desenvolvimento em abordagem incremental, com sprints semanais de levantamento, implementação, testes e documentação.
- Implementar e validar, na segunda etapa, o cadastro, a listagem, a pesquisa, a atualização de situação e o resumo geral, com tratamento de entradas inválidas.

3 JUSTIFICATIVA

Socialmente, o projeto melhora a gestão de uma política pública que atende à rede pública de ensino. Quando a equipe visualiza rapidamente o que está planejado, o que já foi feito e qual o percentual de participação alcançado, ganha condições de redistribuir esforços, identificar escolas com baixa adesão e substituir palestras esporádicas por um programa contínuo — o oposto do que Sousa, Esperidião e Medina (2017) apontam como fragilidade. Vale destacar que o sistema mantém cadastro genérico: a temática é apenas um campo do registro, de modo que a mesma aplicação atenda vacinação, saúde bucal ou saúde mental sem alteração de código.

Academicamente, o projeto articula os conteúdos do segundo semestre: estruturas condicionais, laços, vetores, struct e, principalmente, a divisão do problema em funções com responsabilidades claras — princípio que Sommerville (2018) associa à manutenibilidade do software. A escolha da linguagem C e da interface em terminal mantém o trabalho no nível do curso e concentra o esforço naquilo que a atividade avalia: lógica, modularização e validação de entradas. Por fim, o trabalho exige uma decisão ética explícita sobre o que não deve ser armazenado, exercitando na prática o princípio da necessidade previsto na LGPD (BRASIL, 2018).

4 ENGENHARIA DE SOFTWARE

4.1 Usuários e delimitação do escopo

Quadro 1 — Usuários do sistema

Usuário	Como usa o sistema
Coordenador do PSE (GTI-M)	Cadastra as ações pactuadas, acompanha o andamento nas escolas e usa o resumo geral para prestar contas. É o principal interessado nos indicadores.
Profissional da Atenção Primária	Executa a ação na escola e, logo depois, atualiza a situação para "realizada" e informa a quantidade efetiva de participantes.
Representante da escola	Consulta as ações programadas para a sua unidade, confere datas e público-alvo e comunica cancelamentos.

Está no escopo:

- Cadastro de ações com código, escola, tema, data prevista, público-alvo, responsável e quantidade prevista de participantes.
- Listagem organizada de todos os registros e pesquisa por código, escola ou tema.
- Atualização da situação para planejada, realizada ou cancelada, com registro dos participantes efetivos quando realizada.
- Resumo geral com contagem por situação, total de participantes e percentual de participação.
- Validação de código duplicado, campos vazios, quantidades negativas, opções inexistentes no menu e limite de armazenamento.

Está fora do escopo:

- Qualquer dado individual de estudante: nome, matrícula, prontuário, diagnóstico ou condição clínica. O sistema não possui campos para isso.
- Diagnóstico, triagem médica, prescrição ou recomendação de tratamento.
- Banco de dados, interface gráfica, aplicação web e integração com sistemas oficiais como o e-SUS APS.
- Persistência em arquivo — os dados ficam em memória durante a execução; gravar em arquivo é melhoria opcional.

Esses limites são requisitos de projeto, verificáveis na implementação, e não recomendações. O campo "responsável" registra o profissional que conduz a ação, informação funcional e não sensível. O percentual de participação é indicador de cobertura da ação, nunca de desempenho de pessoas.

4.2 Requisitos funcionais e não funcionais**Quadro 2 — Requisitos funcionais**

ID	Descrição
RF01	Cadastrar ação com código, escola, tema, data prevista, público-alvo, responsável e quantidade prevista, atribuindo a situação inicial PLANEJADA.
RF02	Listar todas as ações cadastradas em formato organizado, informando quando não houver registros.
RF03	Pesquisar ações por código, por escola ou por tema, avisando quando nenhum resultado for encontrado.
RF04	Atualizar a situação da ação para PLANEJADA, REALIZADA ou CANCELADA, localizando o registro pelo código.
RF05	Registrar a quantidade efetiva de participantes quando a situação for alterada para REALIZADA.
RF06	Gerar resumo geral com o número de ações por situação, o total de participantes e o percentual de participação.
RF07	Validar as entradas: código duplicado ou não positivo, campos obrigatórios vazios, quantidades negativas, opções inexistentes e vetor cheio.
RF08	Manter o menu principal em execução, por meio de laço de repetição, até que o usuário escolha a opção de encerrar.

Quadro 3 — Requisitos não funcionais

ID	Atributo	Descrição
RNF01	Usabilidade	Mensagens de confirmação e de erro claras, em português, indicando ao usuário o que corrigir.
RNF02	Portabilidade	Uso apenas da biblioteca padrão do C (C99), compilando sem avisos com gcc -Wall no Windows ou no Linux.
RNF03	Manutenibilidade	Solução dividida em funções de responsabilidade única, evitando concentrar a lógica na função main.
RNF04	Confiabilidade	O programa não encerra de forma inesperada diante de entradas inválidas; o buffer de entrada é limpo após cada leitura.
RNF05	Privacidade	A aplicação não dispõe de campos capazes de armazenar dados pessoais ou clínicos de estudantes.

4.3 Riscos

Quadro 4 — Riscos e ações de mitigação

ID	Risco	Impacto	Mitigação
R01	Indisponibilidade de um integrante em semana de sprint	Alto	Distribuir tarefas em pares e manter o repositório atualizado a cada sprint.
R02	Dificuldade com leitura de textos em C (fgets e buffer sujo)	Médio	Concentrar a leitura em uma única função auxiliar e testá-la isoladamente na Sprint 3.
R03	Escopo crescer além do nível do semestre (arquivos, ordenação)	Médio	Tratar como melhoria opcional, só após o escopo mínimo estar testado.
R04	Atraso na gravação do vídeo da 2ª entrega	Alto	Reservar a Sprint 8 só para roteiro, gravação e edição, com o sistema congelado na Sprint 7.

4.4 Processo de desenvolvimento e sprints

O grupo adotará um processo incremental, com entregas semanais. A escolha se justifica porque o escopo mínimo já vem definido pelo enunciado, mas o grupo está em fase inicial de aprendizagem da linguagem C — cenário em que incrementos pequenos e verificáveis reduzem o risco de acumular erros até o fim do prazo. Pressman e Maxim (2016) descrevem o modelo incremental como adequado quando os requisitos centrais são conhecidos desde o início e a equipe precisa entregar funcionalidade utilizável em etapas. Cada sprint dura uma semana, produz uma versão que compila e executa, e encerra com um commit de fechamento no GitHub e a atualização da pasta de documentação.

Quadro 5 — Planejamento das sprints semanais

Sprint	Foco	Entregável
1	Levantamento e contextualização	Fichamento das fontes e contextualização redigida
2	Modelagem e especificação	Documento da 1ª entrega e repositório criado
3	Estrutura base e cadastro	v0.1 — struct, menu e cadastrarAcao() com validações
4	Listagem e pesquisa	v0.2 — listarAcoes() e pesquisarAcoes()
5	Atualização de situação e resumo	v0.3 — escopo mínimo das seis funcionalidades
6	Testes e refinamento	v1.0 estável e registro dos testes
7	Documentação e casos de uso	Diagrama de casos de uso e manual do usuário
8	Apresentação final	Vídeo e repositório final

5 ALGORITMOS E CÓDIGO

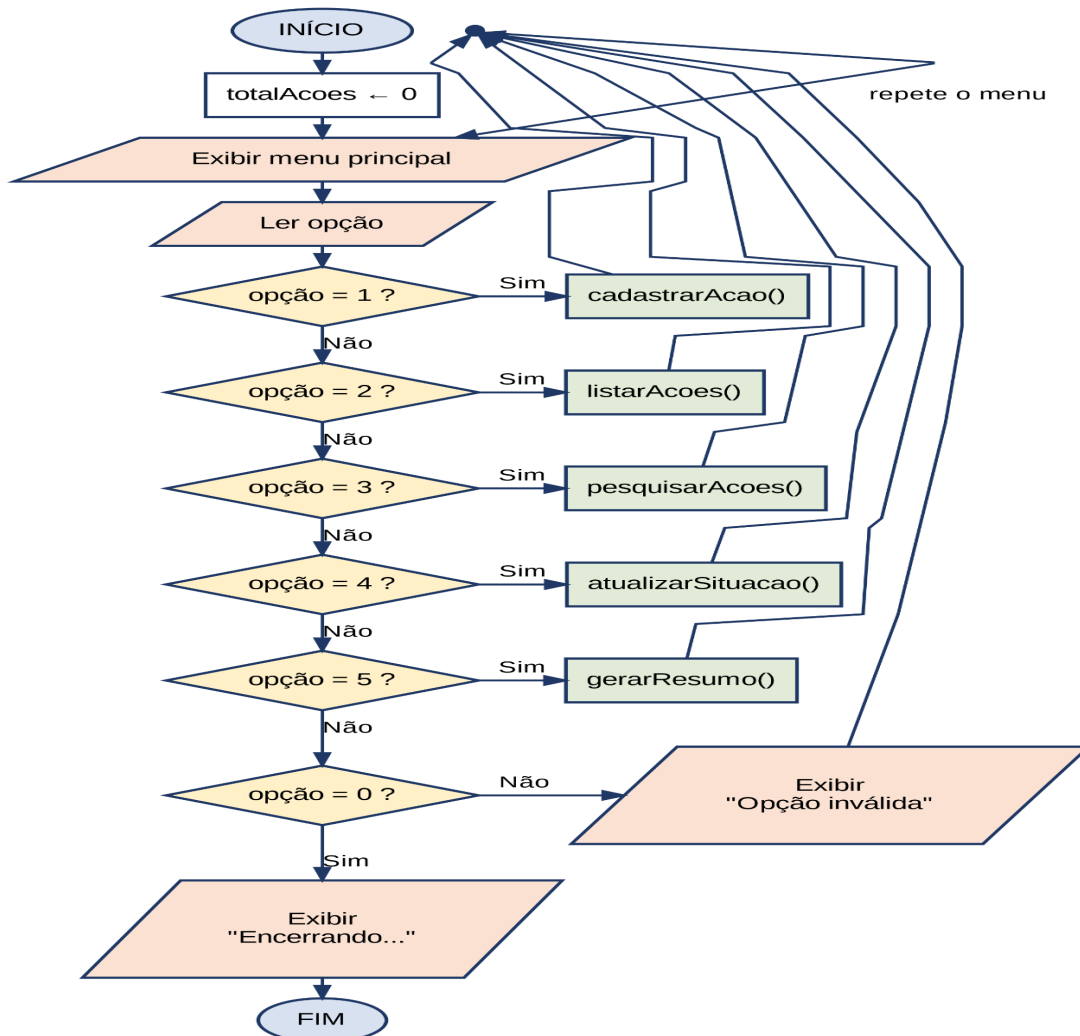
5.1 Estrutura de dados

As ações ficam em um vetor de struct dimensionado por uma constante. Isso atende ao armazenamento em memória exigido pelo enunciado e dispensa alocação dinâmica, mantendo o projeto no nível do semestre. A situação é guardada como número inteiro (0, 1 ou 2) e convertida em texto só na hora de exibir.

5.2 Fluxograma geral do sistema

A Figura 1 mostra o fluxo principal. O programa zera o contador de ações, exibe o menu e testa a opção digitada em cadeia; cada opção válida chama a função responsável, que devolve o controle ao menu. Opções fora da faixa produzem mensagem de erro sem encerrar o programa, e o laço só termina quando o usuário digita 0.

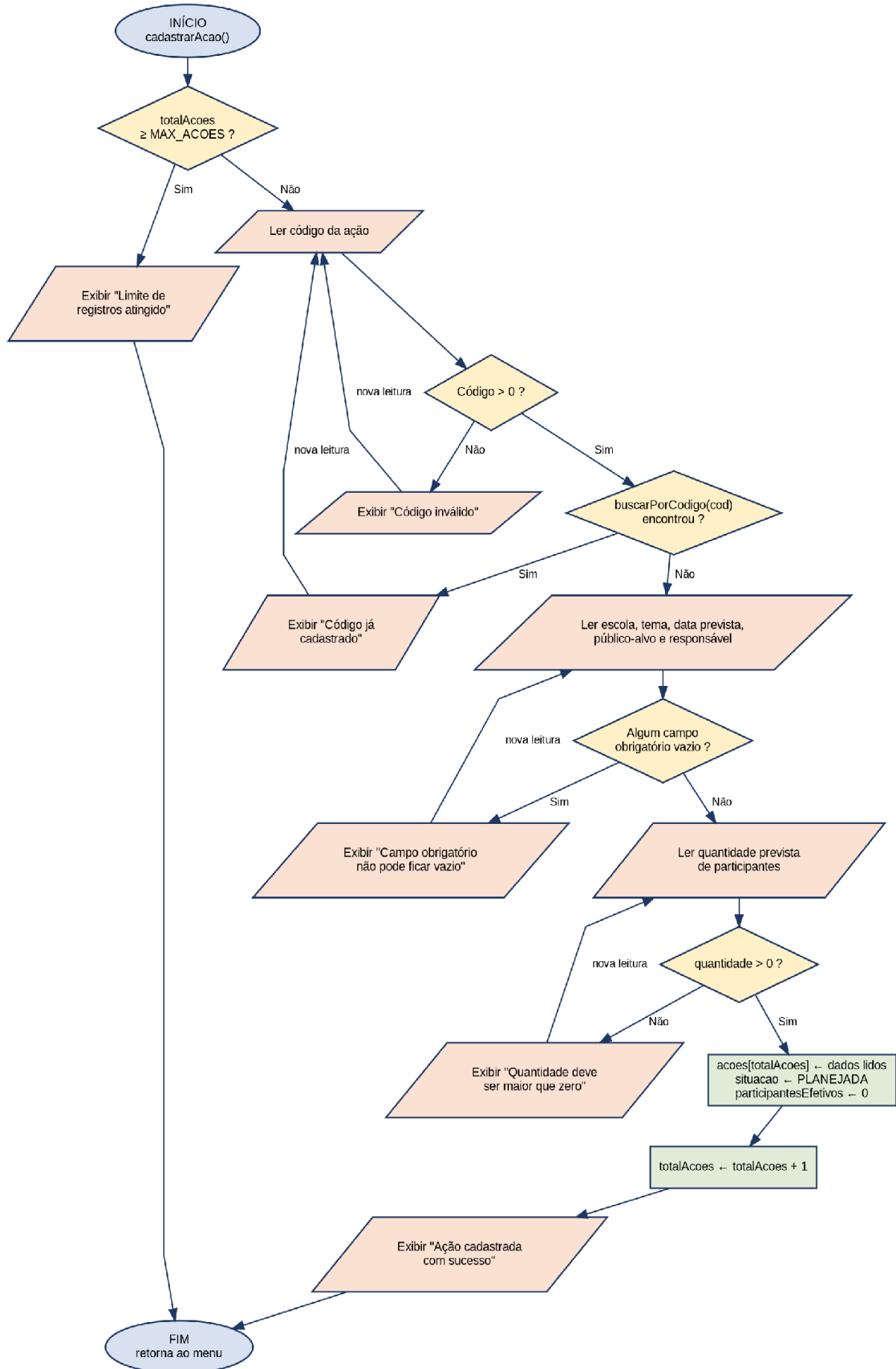
Figura 1 — Fluxograma geral do sistema



5.3 Fluxograma do cadastro de ação

O cadastro foi escolhido para detalhamento por concentrar o maior número de validações. A Figura 2 mostra que a verificação de vetor cheio vem antes de qualquer leitura e que o código é validado quanto ao valor e à duplicidade antes de os demais campos serem pedidos — assim o usuário não preenche o formulário inteiro para só então descobrir que o identificador já existe. Cada validação reprovada volta à leitura correspondente, e não ao início da operação.

Figura 2 — Fluxograma do cadastro de ação



5.4 Pseudocódigo e algoritmos em linguagem C

O pseudocódigo abaixo resume o laço do menu principal em linguagem estruturada. Na sequência, os trechos traduzem os fluxogramas para a linguagem C, usando apenas os recursos estudados no semestre: struct, vetores, laços, condicionais e funções. Eles são o esqueleto da implementação que será concluída na segunda etapa, quando entram também `listarAcoes()`, `pesquisarAcoes()` e `atualizarSituacao()`.

Pseudocódigo 1 — Laço do menu principal (RF08)

```
ALGORITMO PSE_EM_ACAO
INICIO
    totalAcoes <- 0
    REPITA
        ESCRIVA menu com as opcoes de 1 a 5 e 0 para sair
        LEIA opcao
        SE opcao = 1 ENTAO cadastrarAcao()
        SENAO SE opcao = 2 ENTAO listarAcoes()
        SENAO SE opcao = 3 ENTAO pesquisarAcoes()
        SENAO SE opcao = 4 ENTAO atualizarSituacao()
        SENAO SE opcao = 5 ENTAO gerarResumo()
        SENAO SE opcao = 0 ENTAO ESCRIVA "Encerrando o sistema..."
        SENAO ESCRIVA "Opcao invalida! Digite um numero de 0 a 5."
        FIM SE
    ATE QUE opcao = 0
FIM
```

Código 1 — Constantes, estrutura de dados e variáveis globais

```
#include <stdio.h>
#include <string.h>

#define MAX_ACOES 100
#define TAM 60

#define PLANEJADA 0
#define REALIZADA 1
#define CANCELADA 2

typedef struct {
    int    codigo;
    char   escola[TAM];
    char   tema[TAM];
    char   data[15];           /* DD/MM/AAAA */
    char   publico[TAM];
    char   responsavel[TAM];
    int    previstos;
    int    efetivos;
    int    situacao;
} Acao;

Acao acoes[MAX_ACOES];
int totalAcoes = 0;
```

Código 2 — Função main com o menu principal (RF08)

```

int main() {
    int opcao;

    do {
        printf("\n===== PSE EM ACAA =====\n");
        printf("1 - Cadastrar acao\n");
        printf("2 - Listar acoes\n");
        printf("3 - Pesquisar acoes\n");
        printf("4 - Atualizar situacao\n");
        printf("5 - Gerar resumo geral\n");
        printf("0 - Sair\n");
        printf("Opcao: ");

        if (scanf("%d", &opcao) != 1) {
            if (feof(stdin)) {
                opcao = 0;          /* entrada encerrada */
            } else {
                opcao = -1;        /* usuario digitou letra */
            }
        }
        limparEntrada();

        if (opcao == 1) {
            cadastrarAcao();
        } else if (opcao == 2) {
            listarAcoes();
        } else if (opcao == 3) {
            pesquisarAcoes();
        } else if (opcao == 4) {
            atualizarSituacao();
        } else if (opcao == 5) {
            gerarResumo();
        } else if (opcao == 0) {
            printf("Encerrando o sistema...\n");
        } else {
            printf("Opcao invalida! Digite um numero de 0 a 5.\n");
        }
    } while (opcao != 0);

    return 0;
}

```

Código 3 — Funções auxiliares de leitura e busca (RNF03, RNF04)

```

void limparEntrada() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

void lerTexto(char destino[], int tamanho, char rotulo[]) {
    int i;
    do {
        printf("%s: ", rotulo);
        fgets(destino, tamanho, stdin);

        for (i = 0; destino[i] != '\0'; i++) {
            if (destino[i] == '\n') {
                destino[i] = '\0';
            }
        }

        if (strlen(destino) == 0) {
            printf("Campo obrigatorio! Nao pode ficar vazio.\n");
        }
    } while (strlen(destino) == 0);
}

int buscarPorCodigo(int codigo) {
    int i;
    for (i = 0; i < totalAcoes; i++) {
        if (acoes[i].codigo == codigo) {
            return i;
        }
    }
    return -1;
}

```

Código 4 — Cadastro de ação com validações (RF01, RF07)

```

void cadastrarAcao() {
    int codigo, previstos;

    if (totalAcoes >= MAX_ACOES) {
        printf("Limite de registros atingido!\n");
        return;
    }

    do {
        printf("Codigo da acao: ");
        scanf("%d", &codigo);
        limparEntrada();

        if (codigo <= 0) {
            printf("Codigo invalido! Digite um numero positivo.\n");
        } else if (buscarPorCodigo(codigo) != -1) {
            printf("Codigo ja cadastrado! Digite outro.\n");
        }
    } while (codigo <= 0 || buscarPorCodigo(codigo) != -1);

    acoes[totalAcoes].codigo = codigo;
    lerTexto(acoes[totalAcoes].escola, TAM, "Escola");
    lerTexto(acoes[totalAcoes].tema, TAM, "Tema");
    lerTexto(acoes[totalAcoes].data, 15, "Data prevista");
    lerTexto(acoes[totalAcoes].publico, TAM, "Publico-alvo");
    lerTexto(acoes[totalAcoes].responsavel, TAM, "Responsavel");

    do {
        printf("Participantes previstos: ");
        scanf("%d", &previstos);
        limparEntrada();

        if (previstos <= 0) {
            printf("A quantidade deve ser maior que zero!\n");
        }
    } while (previstos <= 0);

    acoes[totalAcoes].previstos = previstos;
    acoes[totalAcoes].efetivos = 0;
    acoes[totalAcoes].situacao = PLANEJADA;
    totalAcoes++;

    printf("Acao cadastrada com sucesso!\n");
}

```

Código 5 — Resumo geral e percentual de participação (RF06)

```

void gerarResumo() {
    int i;
    int planejadas = 0, realizadas = 0, canceladas = 0;
    int totalPrevistos = 0, totalEfetivos = 0;
    float percentual;

    for (i = 0; i < totalAcoes; i++) {
        if (acoes[i].situacao == PLANEJADA) {
            planejadas++;
        } else if (acoes[i].situacao == REALIZADA) {
            realizadas++;
            totalPrevistos = totalPrevistos + acoes[i].previstos;
            totalEfetivos = totalEfetivos + acoes[i].efetivos;
        } else {
            canceladas++;
        }
    }

    printf("\n----- RESUMO GERAL ----- \n");
    printf("Acoes cadastradas: %d\n", totalAcoes);
    printf("Planejadas: %d | Realizadas: %d | Canceladas: %d\n",
        planejadas, realizadas, canceladas);
    printf("Participantes previstos: %d\n", totalPrevistos);
    printf("Participantes efetivos: %d\n", totalEfetivos);

    if (totalPrevistos > 0) {
        percentual = (totalEfetivos * 100.0) / totalPrevistos;
        printf("Percentual de participacao: %.1f%%\n", percentual);
    } else {
        printf("Percentual: nao aplicavel (nenhuma acao realizada).\n");
    }
}

```

Duas decisões merecem registro. A primeira: o percentual de participação soma os previstos apenas das ações já realizadas — somar todas as ações cadastradas produziria um percentual artificialmente baixo, penalizado por atividades que ainda não ocorreram. A segunda: o teste `totalPrevistos > 0` protege contra divisão por zero quando nenhuma ação foi realizada até o momento da consulta.

6 CONSIDERAÇÕES PARCIAIS

Esta primeira etapa concentrou-se em entender o problema antes de codificá-lo. O estudo do PSE mostrou que a dificuldade da equipe fictícia IntegraPSE não é uma invenção didática: a literatura aponta de forma convergente a fragilidade do registro e do monitoramento como um dos entraves do programa. A modelagem também tornou explícitas decisões que só apareceriam durante a codificação, a verificação do código duplicado antes da leitura dos demais campos, a base de cálculo do percentual e a proteção contra denominador nulo. Por fim, a delimitação ética foi tratada como requisito, e não como advertência final: o sistema simplesmente não tem campos capazes de guardar dados pessoais ou clínicos de estudantes.

REFERÊNCIAS

BRASIL. Decreto nº 6.286, de 5 de dezembro de 2007. Institui o Programa Saúde na Escola — PSE e dá outras providências. Diário Oficial da União: seção 1, Brasília, DF, 6 dez. 2007.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: seção 1, Brasília, DF, 15 ago. 2018.

BRASIL. Ministério da Saúde. O que é o Programa Saúde na Escola (PSE)? Brasília, DF, 2025a. Disponível em: <https://www.gov.br/saude/pt-br/composicao/saps/pse>. Acesso em: 10 set. 2026.

BRASIL. Ministério da Saúde. Quais são as ações do PSE? Brasília, DF, 2025b. Disponível em: <https://www.gov.br/saude/pt-br/composicao/saps/pse/acoes>. Acesso em: 10 set. 2026.

BRASIL. Ministério da Saúde; Ministério da Educação. Passo a passo PSE: Programa Saúde na Escola — tecendo caminhos da intersetorialidade. Brasília, DF: Ministério da Saúde, 2011. 46 p. Disponível em: https://bvsms.saude.gov.br/bvs/publicacoes/passo_a_passo_programa_saude_escola.pdf. Acesso em: 10 set. 2026.

IBGE — INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. Pesquisa Nacional de Saúde do Escolar: 2019. Rio de Janeiro: IBGE, 2021. Disponível em: <https://biblioteca.ibge.gov.br/visualizacao/livros/liv101852.pdf>. Acesso em: 10 set. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SOUSA, Marta Caires de; ESPERIDIÃO, Monique Azevedo; MEDINA, Maria Guadalupe. A intersetorialidade no Programa Saúde na Escola: avaliação do processo político-gerecncial e das práticas de trabalho. Ciência & Saúde Coletiva, Rio de Janeiro, v. 22, n. 6, p. 1781-1790, 2017. DOI: 10.1590/1413-81232017226.24262016.